

安全漏洞审计报告 · 第6轮

用户信息管理平台

CSRF 跨站请求伪造漏洞审计报告

User Management Platform — Cross-Site Request Forgery Assessment

目标系统：用户信息管理平台 (Flask)

报告日期：2026-07-14

漏洞等级：严重 × 1 | 高危 × 1

修复状态：✅ 全部已修复

目录

1. 报告概述
2. 漏洞详情
3. 攻击演示
4. 修复方案
5. 修复前后对照
6. 修复验证
7. 全部六轮汇总

1. 报告概述

1.1 本轮背景

系统新增了密码修改功能（/change-password），由于不需要原密码、不需要 CSRF Token、不验证请求来源，导致严重的**CSRF**（跨站请求伪造）漏洞。攻击者只需诱导已登录用户点击一个链接或按钮，即可在用户不知情的情况下修改其密码，完全接管账号。

1.2 什么是 CSRF？

CSRF（Cross-Site Request Forgery） 攻击者伪造请求，利用受害者已登录的 session，在受害者不知情的情况下执行恶意操作（如修改密码、转账）。浏览器会自动携带目标站点的 Cookie，服务端无法区分请求是用户主动发起还是被伪造的。

2. 漏洞详情

1

● 严重

1

● 高危

2/2

✓ 已修复

VULN-21 : 修改密码无 CSRF 防护

● 严重

位置 : /change-password — POST 路由

漏洞代码 :

```
@app.route("/change-password", methods=["POST"])
def change_password():
    # ✗ 没有 CSRF Token 校验
    # ✗ 不需要原密码
    # ✗ 不验证 Referer
    username = request.form.get("username") // 任何已登录用户可改任何人
    new_password = request.form.get("new_password")
```

描述 : 攻击者搭建恶意页面, 内嵌一个自动提交到 /change-password 的表单。受害者只要恰好在登录状态, 浏览器自动携带 session Cookie, 密码即在毫秒间被篡改。

VULN-22 : 充值接口无 CSRF 防护

● 高危

位置 : /recharge — POST 路由

描述 : 同样没有 CSRF Token 校验, 攻击者可构造表单让受害者在不知情下充值或扣款。虽然已限制金额上限和余额非负, 但攻击者可通过多次小额扣款耗尽用户余额。

3. 攻击演示

3.1 攻击者构造钓鱼页面

📌 攻击者在自己的网站上放置以下 **HTML**

```
<form action="http://目标网站/change-password" method="POST">
  <input type="hidden" name="username" value="admin">
  <input type="hidden" name="new_password" value="hacked">
  <button>🧧 点击领取红包</button>
</form>
```

🧐 受害者点击后发生了什么？

1. 浏览器自动携带了目标站点的 session Cookie
2. POST 请求发送到 /change-password
3. 服务端没有 CSRF Token 校验 → 直接通过
4. 密码被改为 "hacked"
5. 攻击者用新密码登录 → 完全接管账号

3.2 实际攻击验证

📌 攻击结果

旧密码 "admin123" 登录 → ❌ 失败

新密码 "hacked_by_csrf" 登录 → ✅ 成功

攻击者成功完全控制了 admin 账号！

4. 修复方案

4.1 新增 CSRF Token 机制

```
import secrets

def generate_csrf_token():
    if "csrf_token" not in session:
        session["csrf_token"] = secrets.token_hex(32)
    return session["csrf_token"]

def validate_csrf_token():
    token = request.form.get("csrf_token")
    stored = session.get("csrf_token")
    return token and stored and token == stored
```

4.2 在 POST 路由中校验

```
if not validate_csrf_token():
    return redirect("/profile?msg=" + quote("CSRF Token 无效"))
```

4.3 在表单中添加隐藏字段

```
# 模板中 :
<input type="hidden" name="csrf_token" value="{ csrf_token }">
```

5. 修复前后对照

✖ 修复前

```
<form action="/change-password">  
  username + new_password  
</form>
```

攻击者从外部提交:

✔ 直接通过

→ CSRF 攻击成功

✔ 修复后

```
<form action="/change-password">  
  username + new_password + csrf_token  
</form>
```

攻击者从外部提交 (无 Token) :

✖ 被拒绝

→ CSRF 攻击失败

6. 修复验证

#	测试	结果
1	不带 CSRF Token 修改密码 → 被拒绝	✓ 通过
2	带 CSRF Token 修改密码 → 成功	✓ 通过
3	不带 CSRF Token 充值 → 被拒绝	✓ 通过
4	带 CSRF Token 充值 → 成功	✓ 通过

7. 全部六轮汇总

轮次	审计内容	漏洞数	严重	高危	中危	状态
第 1 轮	密码安全	8	4	-	4	✓
第 2 轮	SQL 注入	2	2	-	-	✓
第 3 轮	文件上传	1	1	-	-	✓
第 4 轮	业务逻辑	7	2	2	3	✓
第 5 轮	文件包含	2	2	-	-	✓
第 6 轮	CSRF	2	1	1	-	✓
合计		22	12	3	7	全部已修复

安全建议

- 所有 **POST** 请求都必须有 **CSRF** 防护：修改密码、充值等敏感操作尤其重要
- 修改密码必须验证原密码：即使有 CSRF Token，也应要求用户输入原密码
- 不要信任浏览器自动携带的 **Cookie**：CSRF 正是利用了这一特性
- 使用 **SameSite Cookie**：设置 SameSite=Lax 可防止大部分跨站请求携带 Cookie

安全漏洞审计报告 · 第6轮 — CSRF 安全 | 报告日期：2026-07-14

项目地址：<https://github.com/sencly-book/user-management>